Group Signatures with Message-Dependent Opening Directly Imply Timed-Release Encryption

Yuto Imura[§] and Keita Emura*[§,†]

[§]Kanazawa University, Japan.
[†]National Institute of Advanced Industrial Science and Technology, Japan.

July 25, 2025

Abstract

Group signatures (GS, Chaum and van Heyst, EUROCRYPT 1991) are digital signatures that allow a signer to anonymously prove the membership and also allow the special authority called the opener can identify the signer. Group signatures with message-dependent opening (GS-MDO, Sakai et al., Pairing 2012) weakened the power of the opener by introducing another authority called the admitter who issues a message-dependent token. It would be a natural research topic to clarify whether cryptographic primitives that are required to construct GS-MDO are stronger than those of GS or not, according to the enhanced functionality of GS-MDO. In this paper, we propose a generic construction of timed-release encryption (TRE) from GS-MDO. Note that Sakai et al. have shown that GS-MDO implies identity-based encryption (IBE), and Nakai et al. (IWSEC 2009) and Matsuda et al. (Pairing 2010) demonstrated generic constructions of TRE from IBE. Thus, we do not show any new result from the viewpoint of feasibility. We show that (1) GS-MDO *directly* implies TRE without employing the generic constructions of TRE from IBE, and (2) the proposed TRE construction provides public verifiability, that is not usually supported by TRE, because a TRE ciphertext is a group signature in our construction. We also introduce a new security notion which we call token unforgeability where no adversary can forge a token even the adversary has the opener's secret key, and prove that token unforgeability is implied by opener anonymity which is a fundamental security notion of GS-MDO. Our result implies that GS-MDO is a very strong cryptographic primitive.

1 Introduction

Group signatures (GSs) [17] are digital signatures that allow a signer to anonymously prove the membership and also allow the special authority called the opener can identify the signer.

1.1 Feasibility of Group Signatures

Bellare, Micciancio, and Warinschi (BMW) introduced a formal definition of Group signatures in [8] which we call the BMW model. The model contains full anonymity and full traceability as fundamental security definitions of GS. They also showed that GS secure in the BMW model can be generically constructed from public key encryption (PKE), signatures, and non-interactive

*Corresponding Author: k-emura@se.kanazawa-u.ac.jp

zero-knowledge proof (NIZK). Security models providing enhanced security/functionality (Bellare-Shi-Zhang (BSZ) [9] and fully dynamic GS [14]) also can be generically constructed from the same building blocks.

As the opposite direction, it has been shown that fully anonymous GS implies PKE [1, 8]. Moreover, the GS-based PKE provides public verifiability, that is not usually supported by PKE, because a PKE ciphertext is a group signature in the GS-based PKE construction. Concretely, Ohtake et al. [45] mentioned that "*It remarks that the PKE scheme, which is constructed with the above method, has public verifiability. Namely, anyone can verify validity of a ciphertext by using* gpk". Here, the above method means the GS-based PKE [1] construction explained in Section 4. As a similar result, in [28], it has been shown that dynamic GS [9,48] implies public key encryption with non-interactive opening (PKENO) [22, 30]. These results suggest that GSs with additional functionalities may imply cryptographic primitives with enhanced functionalities, and claim that GS is a strong cryptographic primitive.

1.2 GS-MDO

Since the opener can freely open signatures in GS, group signatures with message-dependent opening (GS-MDO) [47] have been proposed to weakened the power of the opener by introducing the other authority called the admitter. The signer of a group signature on the message M is identified by using both the token t_{M} (associated to M) issued by the admitter and the opener's secret key ok. There are two anonymity notions, opener anonymity and admitter anonymity that guarantee anonymity from the viewpoint of the opener and the admitter, respectively. As concrete constructions, pairing-based GS-MDO schemes [5, 36, 44, 47] and lattice-based GS-MDO schemes [18, 37] have been proposed so far.

The following example explains well the functionality of GS-MDO. A bidder of an auction signs the bidding price using a GS-MDO and sends the group signature and the bidding price to the admitter. The admitter checks whether a bidder is a legitimate user of the auction or not via the verification algorithm. Due to admitter anonymity, the admitter cannot identify users. Here, we simply assume that the user who bids the highest price is the winner. The admitter generates a token associated to the highest price, and sends the token to the opener. The opener identifies who the winner of the auction is. Due to opener anonymity, the opener cannot identify losers.

GS-MDO examines the functionality that allows user anonymity to be revealed based on the content. As a related approach, set pre-constrained (SPC) group signatures [7, 43] have been proposed in an End-to-End (E2EE) secure messaging context. The key difference between GS-MDO and SPC-GS lies in the timing of when constrained messages are determined. GS-MDO primarily assumes that the admitter selects a message after a user generates a group signature (of course, the admitter can generate a token for a message before a user generates a group signature on the message). Consequently, GS-MDO is well-suited for indicating messages in an ad-hoc manner, such as in anonymous auctions since the winning bid will be determined after generating group signatures. In contrast, SPC-GS defines content in a pre-constrained manner.

1.3 Our Motivation

As a natural question,

Are cryptographic primitives that are required to construct GS-MDO stronger than those of GS or not, according to the enhanced functionality of GS-MDO?

Sakai et al. [47] gave an answer to this question where GS-MDO implies identity-based encryption (IBE). Here, Boneh et al. [13] showed that there is no blackbox construction from PKE to IBE, and

the construction of fully anonymous GS requires PKE as its building block. These results suggest that GS-MDO is stronger than GS.[1]

We pay attention to the fact that Sakai et al. [47] employed only opener anonymity to construct IBE from GS-MDO, and did not employ admitter anonymity which is one of the fundamental security properties in GS-MDO. This suggests that we directly construct an advanced cryptographic primitive from GS-MDO by employing admitter anonymity. Clarifying a security property or functionality, that GS-MDO implicitly-but-originally achieved, strengthens the previous result where GS-MDO implies IBE.

1.4 Our Contribution

In this paper, we propose a generic construction of timed-release encryption (TRE) (e.g., [20, 23, 34, 40, 42, 46]) from GS-MDO. In TRE, a time server broadcasts a time-dependent token. In our construction, we regard the admitter as the time server and the opener as the user (decryptor). Due to *both* admitter anonymity and opener anonymity, a group signature whose signed message is the time T can be identified by using *both* the corresponding time-dependent token and the opening key ok. Our result well explains that the GS-MDO functionality is strongly related to those of TRE. We also introduce a new security notion which we call token unforgeability where no adversary can forge a token even the adversary has ok. This security notion is quite natural in the GS-MDO context because it guarantees that the opener cannot identify a signer if the opener does not obtain the corresponding token from the admitter. We prove that token unforgeability is implied by opener anonymity. That is, we can say that token unforgeability is inherent in GS-MDO.

We note that Nakai et al. [42] and Matsuda et al. [40] have demonstrated generic constructions of TRE from IBE. Since GS-MDO implies IBE as mentioned above, we do not show any new result from the viewpoint of feasibility. Our result implies that GS-MDO is a very strong cryptographic primitive because:

1. We show that GS-MDO *directly* implies TRE without employing the generic constructions of TRE from IBE [40, 42].

2. Because a TRE ciphertext is a group signature in the proposed TRE construction, it provides public verifiability, that is not usually supported by TRE.

On Generic Constructions of TRE from IBE. Note that in the Nakai et al.'s TRE construction [42], the randomness (used for generating an IBE ciphertext) is contained in a TRE ciphertext. Thus, the construction does not provide public verifiability even if the building blocks (IBE and PKE) are publicly verifiable. Matsuda et al's TRE construction [40] employed the KEM/DEM framework (where KEM stands for key encapsulation mechanism and DEM stands for data encapsulation mechanism) and employed IBE-KEM, tag-KEM, and DEM as building blocks. Though CCA-secure tag-KEM can be constructed from CCA-secure KEM and one-time secure message authentication codes (MACs) as mentioned in [2], the MAC part does not provide public verifiability. Thus, due to the above observation, we conclude that these TRE constructions are not publicly verifiable even when the PKE part and the IBE part are publicly verifiable, e.g., they are

[1] To simplify the discussion, we ignore the NIZK part. Actually, it is non-trivial to construct NIZKs proving a NP-relation induced by IBE. For example, Libert and Joye [36] proposed a partially structure-preserving IBE scheme to employ the Groth-Sahai proofs [33] for constructing a GS-MDO scheme in the standard model. This is caused by the non-triviality of the construction of IBE-friendly NIZKs. Moreover, some non-blackbox IBE constructions from PKE-related tools have been proposed [25, 26, 32]. Thus, GS-MDO might be constructed from GS in a non-blackbox manner. For these reasons, we chose to use the word "suggest" in the above explanations.

constructed from GS(-MDO). Note that their constructions consider pre-open capability. Thus, there is a room for providing public verifiability by removing components for providing pre-oepn capability.

1.5 Differences from the proceedings version

An extended abstract appeared at the 24th International Conference on Cryptology and Network Security (CANS) 2025. In this full version, we introduce a new security notion which we call token unforgeability, and prove that token unforgeability is implied by opener anonymity (Section 6). Clarifying a security property, that GS-MDO originally achieved only implicitly, can be evidence that GS-MDO is a strong cryptographic primitive. This additional result strengthens the result given in the proceedings version.

2 GS-MDO: Group Signatures with Message-Dependent Opening

2.1 Syntax of GS-MDO

In this section, we define GS-MDO. Note that the opening key ok and the admitter's key ask are generated by the same algorithm, GKg, in the original definition given by Sakai et al. [47]. In this paper, we divide these key generation processes since it is assumed that two authorities (the opener and the admitter) do not collude with each other. Moreover, we introduce the setup algorithm $\mathsf{GS\text{-}MDO.Setup}$ to support multiple users in the proposed TRE construction. That is, we can capture the case that the GKg algorithm is run multiple times (i.e., there are multiple openers), whereas the GKg algorithm is usually run only once in the GS-MDO context. We also remove the group public key gpk from the input of the token generation algorithm Td. Then, we can capture the situation that a token does not depend on gpk (user in the TRE context) and is commonly used for all users. For example, the Ohara et al.'s pairing-based GS-MDO scheme [44] and the Libert et al.'s lattice-based GS-MDO scheme [37] can be modified to match the syntax (See the Appendices A and B). That is, our syntax modification is a natural requirement and is not a restriction. We also note that the security of an opener must be preserved even if other openers are compromised. Since the number of openers is bounded by a polynomial in the security parameter, GS-MDO in the single-opener setting and in the multi-opener setting are essentially equivalent, provided that a polynomial reduction loss is acceptable. Therefore, we define GS-MDO in the single-opener setting here and explain the definition for the multi-opener setting in the Appendix C.1.

A GS-MDO scheme $\mathsf{GS\text{-}MDO}$ consists of seven algorithms ($\mathsf{GS\text{-}MDO.Setup}, \mathsf{GKg}, \mathsf{AKg}, \mathsf{GSig}, \mathsf{Td}, \mathsf{GVf}, \mathsf{Open}$) defined as follows.

Definition 1 (Syntax of GS-MDO)**.** $\mathsf{GS\text{-}MDO.Setup}$**:** *The setup algorithm takes a security parameter λ as input, and outputs a common parameter $\mathsf{pp}_{\mathsf{GS\text{-}MDO}}$. We implicitly assume that all algorithms take $\mathsf{pp}_{\mathsf{GS\text{-}MDO}}$ as input.*

GKg**:** *The group key generation algorithm takes $\mathsf{pp}_{\mathsf{GS\text{-}MDO}}$ and the number of group members $n \in \mathbb{N}$ as input, and outputs a group public key gpk, an opening key ok, and n group signing keys $\{\mathsf{gsk}_i\}_{i\in[n]}$.*

AKg**:** *The admitter key generation algorithm takes $\mathsf{pp}_{\mathsf{GS\text{-}MDO}}$ as input, and outputs an admitter's public key apk and an admitter's secret key ask.*

GSig: *The signing algorithm takes* gpk, apk, gsk_i, *and a message* M *as input, and outputs a group signature* σ.

Td: *The message-dependent token generation algorithm takes* ask *and* M *as input, and outputs a token* t_M.

GVf: *The verification algorithm takes* gpk, apk, M, *and* σ *as input, and outputs 1 (accept) or 0 (reject).*

Open: *The opening algorithm takes* gpk, apk, ok, M, σ, *and* t_M *as input, and outputs a user index* $i \in [n]$ *or* $\perp$.

Correctness is defined as follows. For any $\mathsf{pp}_{\mathsf{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \{\mathsf{gsk}_i\}_{i\in[n]}) \leftarrow \mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^n)$, $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}})$, $\mathsf{M} \in \{0,1\}^*$, and $i \in [n]$, we say that $\mathsf{GS\text{-}MDO}$ is correct if

$$\mathsf{GVf}(\mathsf{gpk}, \mathsf{apk}, \mathsf{M}, \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_i, \mathsf{M})) = 1$$

and

$$\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{M}, \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_i, \mathsf{M}), \mathsf{Td}(\mathsf{ask}, \mathsf{M})) = i$$

hold.

2.2 Security Definitions

Next, we define two anonymity notions, admitter anonymity and opener anonymity. Admitter anonymity guarantees that anonymity holds against an adversary who has the admitter's secret key ask and all signing keys $\{\mathsf{gsk}_i\}_{i\in[n]}$ unless the adversary has the opener's secret key ok, and is defined as follows.

Definition 2 (Admitter Anonymity)**.** *Let* $\mathcal{A}$ *be an adversary and* $\mathcal{C}$ *be the challenger.*

Setup: $\mathcal{C}$ *computes* $\mathsf{pp}_{\mathsf{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \{\mathsf{gsk}_i\}_{i\in[n]}) \leftarrow \mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^n)$, *and* $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}})$, *and gives* $(\mathsf{gpk}, \mathsf{apk}, \mathsf{ask}, \{\mathsf{gsk}_i\}_{i\in[n]})$ *to* $\mathcal{A}$.

Open Query (Phase I): $\mathcal{A}$ *sends* (σ, M) *to* $\mathcal{C}$. *If* M *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{t}_\mathsf{M} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{M})$, *and stores* $(\mathsf{M}, \mathsf{t}_\mathsf{M})$. *Otherwise,* $\mathcal{C}$ *retrieves* t_M. $\mathcal{C}$ *gives the result of* $\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{M}, \sigma, \mathsf{t}_\mathsf{M})$ *to* $\mathcal{A}$.

Challenge: $\mathcal{A}$ *declares* $i_0, i_1 \in [n]$ *and* M^*. $\mathcal{C}$ *chooses* $b \xleftarrow{\$} \{0,1\}$, *computes the challenge group signature* $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{i_b}, \mathsf{M}^*)$, *and gives* σ^* *to* $\mathcal{A}$.

Open Query (Phase II): $\mathcal{A}$ *sends* $(\sigma, \mathsf{M}) \neq (\sigma^*, \mathsf{M}^*)$ *to* $\mathcal{C}$. *If* M *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{t}_\mathsf{M} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{M})$, *and stores* $(\mathsf{M}, \mathsf{t}_\mathsf{M})$. *Otherwise,* $\mathcal{C}$ *retrieves* t_M. $\mathcal{C}$ *gives the result of* $\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{M}, \sigma, \mathsf{t}_\mathsf{M})$ *to* $\mathcal{A}$.

Guess: A *outputs* $b' \in \{0,1\}$.

We say that GS-MDO *provides admitter anonymity if, for all PPT (Probabilistic Polynomial-Time) adversaries* $\mathcal{A}$*, the advantage*

$$\mathsf{Adv}^{\mathsf{admitter\text{-}anon}}_{\mathsf{GS\text{-}MDO},\mathcal{A}}(\lambda, n) = \left|\Pr\left[b' = b\right] - \frac{1}{2}\right|$$

is negligible in the security parameter.

Opener anonymity guarantees that anonymity holds against an adversary who has the opener's secret key ok and all signing keys $\{\mathsf{gsk}_i\}_{i\in[n]}$ unless the adversary has the corresponding token, and is defined as follows.

Definition 3 (Opener Anonymity)**.** *Let* $\mathcal{A}$ *be an adversary and* $\mathcal{C}$ *be the challenger.*

Setup: $\mathcal{C}$ *computes* $\mathsf{pp}_{\mathsf{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$*,* $(\mathsf{gpk}, \mathsf{ok}, \{\mathsf{gsk}_i\}_{i\in[n]}) \leftarrow \mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^n)$*, and* $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}})$*, and gives* $(\mathsf{gpk}, \mathsf{ok}, \mathsf{apk}, \{\mathsf{gsk}_i\}_{i\in[n]})$ *to* $\mathcal{A}$*.*

Token Query (Phase I): $\mathcal{A}$ *sends* M *to* $\mathcal{C}$*. If* M *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{t}_\mathsf{M} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{M})$*, and stores* $(\mathsf{M}, \mathsf{t}_\mathsf{M})$*. Otherwise,* $\mathcal{C}$ *retrieves* t_M*.* $\mathcal{C}$ *gives* t_M *to* $\mathcal{A}$*.*

Challenge: $\mathcal{A}$ *declares* $i_0, i_1 \in [n]$ *and* M^**. Note that* $\mathcal{A}$ *is not allowed to send messages as* M^* *if* $\mathcal{A}$ *has sent these messages as token queries.* $\mathcal{C}$ *chooses* $b \xleftarrow{\$} \{0,1\}$*, computes the challenge group signature* $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{i_b}, \mathsf{M}^*)$*, and gives* σ^* *to* $\mathcal{A}$*.*

Token Query (Phase II): $\mathcal{A}$ *sends* $\mathsf{M} \neq \mathsf{M}^*$ *to* $\mathcal{C}$*. If* M *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{t}_\mathsf{M} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{M})$*, and stores* $(\mathsf{M}, \mathsf{t}_\mathsf{M})$*. Otherwise,* $\mathcal{C}$ *retrieves* t_M*.* $\mathcal{C}$ *gives* t_M *to* $\mathcal{A}$*.*

Guess: $\mathcal{A}$ *outputs* $b' \in \{0,1\}$*.*

We say that GS-MDO *provides opener anonymity if, for all PPT adversaries* $\mathcal{A}$*, the advantage*

$$\mathsf{Adv}^{\mathsf{opener\text{-}anon}}_{\mathsf{GS\text{-}MDO},\mathcal{A}}(\lambda, n) = \left|\Pr\left[b' = b\right] - \frac{1}{2}\right|$$

is negligible in the security parameter.

From these two anonymity notions, GS-MDO guarantees that anonymity holds unless both the opener's secret key and the corresponding token are used.

3 TRE: Timed-Release Encryption

3.1 Syntax of TRE

In this section, we define TRE. We adopt the model from [40,42]. Though they considered additional functionality of TRE which they called pre-open capability, we do not consider it in this paper. We also note that TRE must take into account a multi-user setting. That is, the security of each user must remain intact even if other users are compromised. Given that the number of users is bounded by a polynomial in the security parameter, the single-user and multi-user settings for TRE are essentially equivalent, provided a polynomial reduction loss is acceptable. Therefore, we define TRE in the single-user setting here, and explain the definition for the multi-user setting in the Appendix C.2.

A TRE scheme TRE consists of five algorithms (TRE.Setup, TRE.Ext, TRE.UKg, TRE.Enc, TRE.Dec) defined as follows. We denote m as a plaintext to distinguish a signed message M of GS-MDO.

Definition 4 (Syntax of TRE)**.** $\mathsf{TRE.Setup}$**:** *The setup algorithm takes a security parameter* λ *as input, and outputs a common parameter* $\mathsf{pp_{TRE}}$*. Assume that all algorithms implicitly take* $\mathsf{pp_{TRE}}$ *as input.*

$\mathsf{TRE.TKg}$**:** *The time server key generation algorithm takes* $\mathsf{pp_{TRE}}$ *as input, and outputs a time-server's public key* tpk *and a time-server's secret key* tsk*.*

$\mathsf{TRE.UKg}$**:** *The user key generation algorithm takes* $\mathsf{pp_{TRE}}$ *as input, and outputs a user's public key* $\mathsf{pk_u}$ *and a user's secret key* $\mathsf{sk_u}$*. Assume that* $\mathsf{pk_u}$ *implicitly contains a plaintext space* $\mathsf{MS_{TRE}}$*.*

$\mathsf{TRE.Ext}$**:** *The token generation algorithm takes* tsk *and a time* $\mathsf{T} \in \{0,1\}^*$ *as input, and outputs a token* $\mathsf{s_T}$*.*

$\mathsf{TRE.Enc}$**:** *The encryption algorithm takes* $\mathsf{pk_u}$*,* tpk*,* T*, and a plaintext* $\mathsf{m} \in \mathsf{MS_{TRE}}$ *as input, and outputs a ciphertext* C*.*

$\mathsf{TRE.Dec}$**:** *The decryption algorithm* tpk*,* $\mathsf{sk_u}$*,* $\mathsf{s_T}$*, and* C *as input, and outputs* m *or a failure symbol* $\perp$*.*

Correctness is defined as follows. For any $\mathsf{pp_{TRE}} \leftarrow \mathsf{TRE.Setup}(1^\lambda)$, $(\mathsf{tpk}, \mathsf{tsk}) \leftarrow \mathsf{TRE.TKg}(\mathsf{pp_{TRE}})$, $(\mathsf{pk_u}, \mathsf{sk_u}) \leftarrow \mathsf{TRE.UKg}(\mathsf{pp_{TRE}})$, $\mathsf{m} \in \mathsf{MS_{TRE}}$, and $\mathsf{T} \in \{0,1\}^*$, we say that TRE is correct if

$$\mathsf{TRE.Dec}(\mathsf{tpk}, \mathsf{sk_u}, \mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T}), \mathsf{TRE.Enc}(\mathsf{pk_u}, \mathsf{tpk}, \mathsf{T}, \mathsf{m})) = \mathsf{m}$$

holds.

3.2 Security Definitions

Next, we define two security properties, time server security and insider security. The time server security, which is denoted as $\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$, guarantees that even an adversary who has the time server's secret key tsk cannot obtain any information about the plaintext from the ciphertext without having the user's secret key $\mathsf{sk_u}$. Formally, the $\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$ security is defined as follows.

Definition 5 (Time Server Security ($\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$))**.** *Let* $\mathcal{A}$ *be an adversary and* $\mathcal{C}$ *be the challenger.*

Setup: $\mathcal{C}$ *computes* $\mathsf{pp_{TRE}} \leftarrow \mathsf{TRE.Setup}(1^\lambda)$*,* $(\mathsf{tpk}, \mathsf{tsk}) \leftarrow \mathsf{TRE.TKg}(\mathsf{pp_{TRE}})$*, and* $(\mathsf{pk_u}, \mathsf{sk_u}) \leftarrow \mathsf{TRE.UKg}(\mathsf{pp_{TRE}})$*, and gives* $(\mathsf{tpk}, \mathsf{tsk}, \mathsf{pk_u})$ *to* $\mathcal{A}$*.*

Decryption Query (Phase I): $\mathcal{A}$ *sends* (C, T) *to* $\mathcal{C}$*. If* T *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{s_T} \leftarrow \mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T})$ *and stores* $(\mathsf{T}, \mathsf{s_T})$*. Otherwise,* $\mathcal{C}$ *retrieves* $\mathsf{s_T}$*.* $\mathcal{C}$ *gives the result of* $\mathsf{TRE.Dec}(\mathsf{tpk}, \mathsf{sk_u}, \mathsf{s_T}, \mathsf{C})$ *to* $\mathcal{A}$*.*

Challenge: $\mathcal{A}$ *declares two distinct challenge plaintexts* $\mathsf{m}_0^*, \mathsf{m}_1^* \in \mathsf{MS_{TRE}}$ *and the challenge time* T^**.* $\mathcal{C}$ *chooses* $b \xleftarrow{\$} \{0,1\}$*, computes the challenge cipher text* $C^* \leftarrow \mathsf{TRE.Enc}(\mathsf{tpk}, \mathsf{pk_u}, \mathsf{T}^*, \mathsf{m}_b^*)$ *and gives* C^* *to* $\mathcal{A}$*.*

Decryption Query (Phase II): $\mathcal{A}$ *sends* $(\mathsf{C}, \mathsf{T}) \neq (\mathsf{C}^*, \mathsf{T}^*)$. *If* T *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{s_T} \leftarrow \mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T})$ *and stores* $(\mathsf{T}, \mathsf{s_T})$. *Otherwise,* $\mathcal{C}$ *retrieves* $\mathsf{s_T}$. $\mathcal{C}$ *gives the result of* $\mathsf{TRE.Dec}(\mathsf{tpk}, \mathsf{sk_u}, \mathsf{s_T}, \mathsf{C})$ *to* $\mathcal{A}$.

Guess: $\mathcal{A}$ *outputs* $b' \in \{0, 1\}$.

We say that TRE *provides the* $\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$ *security if, for all PPT adversaries* $\mathcal{A}$*, the advantage*

$$\mathsf{Adv}^{\mathsf{IND\text{-}TR\text{-}CCA_{TS}}}_{\mathsf{TRE},\mathcal{A}}(\lambda) = \left|\Pr\left[b' = b\right] - \frac{1}{2}\right|$$

is negligible in the security parameter.

The insider security, which is denoted as $\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$, guarantees that even an adversary who has the user's secret key $\mathsf{sk_u}$ cannot obtain any information about the plaintext from the ciphertext without having the corresponding token $\mathsf{s}^*_{\mathsf{T}}$ for the (challenge) ciphertext decryption time T^*. Formally, the $\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$ security is defined as follows.

Definition 6 (Insider Security ($\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$))**.** *Let* $\mathcal{A}$ *be an attacker and* $\mathcal{C}$ *be the challenger.*

Setup: $\mathcal{C}$ *computes* $\mathsf{pp_{TRE}} \leftarrow \mathsf{TRE.Setup}(1^\lambda)$, $(\mathsf{tpk}, \mathsf{tsk}) \leftarrow \mathsf{TRE.TKg}(\mathsf{pp_{TRE}})$, *and* $(\mathsf{pk_u}, \mathsf{sk_u}) \leftarrow \mathsf{TRE.UKg}(\mathsf{pp_{TRE}})$, *and gives* $(\mathsf{tpk}, \mathsf{pk_u}, \mathsf{sk_u})$ *to* $\mathcal{A}$.

Extract Query (Phase I): $\mathcal{A}$ *sends* T *to* $\mathcal{C}$. *If* T *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{s_T} \leftarrow \mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T})$ *and stores* $(\mathsf{T}, \mathsf{s_T})$. *Otherwise,* $\mathcal{C}$ *retrieves* $\mathsf{s_T}$. $\mathcal{C}$ *gives* $\mathsf{s_T}$ *to* $\mathcal{A}$.

Challenge: $\mathcal{A}$ *declares two distinct challenge plaintexts* $\mathsf{m}^*_0, \mathsf{m}^*_1 \in \mathsf{MS_{TRE}}$ *and the challenge time* T^* *where* T^* *has not been sent as an extract query.* $\mathcal{C}$ *chooses* $b \xleftarrow{\$} \{0, 1\}$, *computes* $\mathsf{C}^* \leftarrow \mathsf{TRE.Enc}(\mathsf{tpk}, \mathsf{pk_u}, \mathsf{T}^*, \mathsf{m}^*_b)$, *and gives* C^* *to* $\mathcal{A}$.

Extract Query (Phase II): $\mathcal{A}$ *sends* $\mathsf{T} \neq \mathsf{T}^*$. *If* T *is not previously queried, then* $\mathcal{C}$ *computes* $\mathsf{s_T} \leftarrow \mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T})$ *and stores* $(\mathsf{T}, \mathsf{s_T})$. *Otherwise,* $\mathcal{C}$ *retrieves* $\mathsf{s_T}$. $\mathcal{C}$ *gives* $\mathsf{s_T}$ *to* $\mathcal{A}$.

Guess: $\mathcal{A}$ *outputs* $b' \in \{0, 1\}$.

We say that TRE *provides the* $\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$ *security if, for any PPT adversaries* $\mathcal{A}$*, the advantage*

$$\mathsf{Adv}^{\mathsf{IND\text{-}TR\text{-}CPA_{IS}}}_{\mathsf{TRE},\mathcal{A}}(\lambda) = \left|\Pr\left[b' = b\right] - \frac{1}{2}\right|$$

is negligible in the security parameter.

4 Proposed Construction

In this section, we give our proposed generic construction of TRE from GS-MDO. Before giving the proposed construction, we revisited Abdalla et al.'s PKE construction from GS [1] and Sakai et al.'s IBE construction from GS-MDO [47] because we combine these techniques.

GS-based PKE [1]: For a plaintext $\mathsf{m} \in \{0,1\}$, two signing keys $(\mathsf{gsk}_1, \mathsf{gsk}_2)$ are set as the public key of PKE. A ciphertext is a group signature σ generated by gsk_{m+1} (here, a message to be signed is not specifically indicated). Due to full anonymity of GS, information of which signing key is used for generating a group signature is not revealed even all signing keys are made public. The opening key ok is regarded as the decryption key of PKE.

GS-MDO-based IBE [47]: As in the above PKE construction, two signing keys $(\mathsf{gsk}_1, \mathsf{gsk}_2)$ are set as the public parameter. In addition to these signing keys, the opening key ok is also included to the parameter. A ciphertext is a group signature σ generated by gsk_{m+1} where ID is regarded as the message to be signed, i.e., $\sigma \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{m+1}, \mathsf{ID})$. The key generation center has the admitter's secret key ask as the master secret key, and for an identity ID, the center generates a token t_{ID} which is regarded as the decryption key. A ciphertext is decrypted by using the opening algorithm that takes t_{ID} (and ok that is contained in the public parameter). Due to opener anonymity of GS-MDO, information of which signing key is used for generating a group signature is not revealed even all signing keys and ok are made public. We emphasize that admitter anonymity is not employed in the construction.

High-level Description: Our construction is a combination of the above constructions. We set $(\mathsf{gpk}, \mathsf{gsk}_1, \mathsf{gsk}_2)$ as a user's public key pk_u and ok as a user's secret key sk_u. As in the GS-based PKE construction, $(\mathsf{gsk}_1, \mathsf{gsk}_2)$ is contained in the public key. To encrypt a plaintext $\mathsf{m} \in \{0,1\}$, a ciphertext C is a group signature σ generated by gsk_{m+1} where the time T is regarded as the message to be signed, i.e.,

$$\mathsf{C} := \sigma \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{m+1}, \mathsf{T})$$

Since a TRE ciphertext is a group signature on T, C can be publicly verified whether $\mathsf{GVf}(\mathsf{gpk}, \mathsf{apk}, \mathsf{T}, \mathsf{C}) = 1$ holds or not. We note that ok is not contained to a public value unlike the GS-MDO-based IBE construction. The time server has the admitter's secret key ask as the time server's secret key tsk, and generates a token t_T for T. Then, by using both $\mathsf{ok} = \mathsf{sk}_\mathsf{u}$ and t_T, a group signature on T can be opened, i.e., the opening algorithm (the decryption algorithm in the TRE context) identifies which signing key is used. Intuitively, due to admitter anonymity, the time server that has ask does not obtain information of which signing key is used. Due to opener anonymity, a user that has ok does not obtain information of which signing key is used unless the user obtains the corresponding time token.

The proposed generic construction of $\mathsf{TRE} = (\mathsf{TRE.Setup}, \mathsf{TRE.TKg}, \mathsf{TRE.UKg}, \mathsf{TRE.Ext}, \mathsf{TRE.Enc}, \mathsf{TRE.Dec})$ from $\mathsf{GS\text{-}MDO} = (\mathsf{GS\text{-}MDO.Setup}, \mathsf{GKg}, \mathsf{AKg}, \mathsf{GSig}, \mathsf{Td}, \mathsf{GVf}, \mathsf{Open})$ is as follows.

$\mathsf{TRE.Setup}(1^\lambda)$**:** Run $\mathsf{pp}_{\mathsf{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$ and output $\mathsf{pp}_\mathsf{TRE} = \mathsf{pp}_{\mathsf{GS\text{-}MDO}}$.

$\mathsf{TRE.TKg}(\mathsf{pp}_\mathsf{TRE})$**:** Parse $\mathsf{pp}_\mathsf{TRE} = \mathsf{pp}_{\mathsf{GS\text{-}MDO}}$. Run $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}})$ and output $(\mathsf{tpk}, \mathsf{tsk}) = (\mathsf{apk}, \mathsf{ask})$.

$\mathsf{TRE.UKg}(\mathsf{pp}_\mathsf{TRE})$**:** Run $(\mathsf{gpk}, \mathsf{ok}, \mathsf{gsk}_1, \mathsf{gsk}_2) \leftarrow \mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^2)$ and output $\mathsf{pk}_\mathsf{u} = ((\mathsf{gpk}, (\mathsf{gsk}_1, \mathsf{gsk}_2))$ and $\mathsf{sk}_\mathsf{u} = (\mathsf{pk}_\mathsf{u}, \mathsf{ok})$.

$\mathsf{TRE.Ext}(\mathsf{tsk}, \mathsf{T})$**:** Parse $\mathsf{tsk} = \mathsf{ask}$. Run $\mathsf{t}_\mathsf{T} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{T})$ and output $\mathsf{s}_\mathsf{T} = \mathsf{t}_\mathsf{T}$.

$\mathsf{TRE.Enc}(\mathsf{tpk}, \mathsf{pk}_\mathsf{u}, \mathsf{T}, \mathsf{m})$**:** Parse $\mathsf{tpk} = \mathsf{apk}$ and $\mathsf{pk}_\mathsf{u} = (\mathsf{gpk}, (\mathsf{gsk}_1, \mathsf{gsk}_2))$, and let $\mathsf{m} \in \{0,1\}$. Run $\sigma \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{m+1}, \mathsf{T})$ and output $\mathsf{C} = \sigma$.

$\mathsf{TRE.Dec}(\mathsf{tpk}, \mathsf{sk_u}, \mathsf{s_T}, \mathsf{T}, \mathsf{C})$**:** Parse $\mathsf{tpk} = \mathsf{apk}$, $\mathsf{sk_u} = (\mathsf{pk_u}, \mathsf{ok})$, $\mathsf{pk_u} = ((\mathsf{gpk}, (\mathsf{gsk}_1, \mathsf{gsk}_2))$, $\mathsf{s_T} = \mathsf{t_T}$, and $\mathsf{C} = \sigma$. Run $i \leftarrow \mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{T}, \mathsf{t_T}, \sigma)$. Output $\perp$ if $i = \perp$. Otherwise, output $\mathsf{m} = i - 1$

Obviously, the proposed construction is correct if the underlying GS-MDO scheme is correct.

On Multi-bit Encryption: Our TRE construction provides the 1-bit plaintext space. If no CCA security is required, then the plaintext space can be amplified by simply arranging ciphertexts of 1-bit plaintext. To preserve the CCA security, the GS-based PKENO construction [28] uses a message to be signed as a tag, and a verification key $\mathsf{vk_{OTS}}$ of the one-time signature scheme is set as the tag to employ the CHK transformation [16]. That is, a ciphertext of the i-th bit m_i is a group signature generated by $\mathsf{gsk}_{\mathsf{m}_i+1}$ on $\mathsf{vk_{OTS}}$, and all group signatures are signed by the OTS scheme. Unfortunately, this technique cannot be applied directly in GS-MDO-based IBE/TRE since the signed message is ID or T and $\mathsf{vk_{OTS}}$ is chosen on the fly (i.e., in the encryption algorithm). Note that Sakai et al. did not mention how to amplify the message space in their GS-MDO-based IBE construction. How to amplify the plaintext space in GS-MDO-based IBE/TRE is an interesting open problem, e.g., the Myers-Shelat technique [41] might be employed.

5 Security Analysis

In the following proofs, due to the one-bit plaintext space, we set the challenge plaintexts $(\mathsf{m}_0^*, \mathsf{m}_1^*) = (0, 1)$ without loss of generality. Similarly, we set the challenge users $(i_0^*, i_1^*) = (1, 2)$.

Theorem 1. *The proposed construction provides the time server security if the underlying GS-MDO provides admitter anonymity.*

Intuitively, due to admitter anonymity, the admitter that has ask does not obtain information of which signing key is used from group signatures. Since the admitter is set as the time server in our TRE construction, the $\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$ adversary $\mathcal{A}$ that has $\mathsf{tsk} = \mathsf{ask}$ does not obtain information of plaintext from ciphertexts. Let $\mathcal{C}$ be the challenger of admitter anonymity. We construct an algorithm $\mathcal{B}$ that breaks admitter anonymity using $\mathcal{A}$ as follows. Here, $\mathcal{B}$ simulates the decryption oracle using the opening oracle. Note that $\mathcal{A}$ sends a decryption query $(\mathsf{C}, \mathsf{T}) \neq (\mathsf{C}^*, T^*)$, $\mathcal{B}$ can simulate the decryption oracle using the opening oracle where the adversary is not allows to send (σ^*, M^*).

Proof. We give the security proof as follows.

Setup: $\mathcal{C}$ computes $\mathsf{pp_{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \mathsf{gsk}_1, \mathsf{gsk}_2) \leftarrow \mathsf{GKg}(\mathsf{pp_{GS\text{-}MDO}}, 1^2)$, and $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp_{GS\text{-}MDO}})$, and gives $(\mathsf{gpk}, \mathsf{apk}, \mathsf{ask}, \mathsf{gsk}_1, \mathsf{gsk}_2)$ to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{tpk} = \mathsf{apk}$, $\mathsf{tsk} = \mathsf{ask}$, $\mathsf{pk_u} = (\mathsf{gpk}, \mathsf{gsk}_1, \mathsf{gsk}_2)$ and sends $(\mathsf{tpk}, \mathsf{tsk}, \mathsf{pk_u})$ to $\mathcal{A}$.

Decryption Query (Phase I): $\mathcal{A}$ sends (C, T) to $\mathcal{B}$. $\mathcal{B}$ parses $\sigma = \mathsf{C}$ and sends the opening query (σ, T) to $\mathcal{C}$. $\mathcal{C}$ returns $i \leftarrow \mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{T}, \sigma, \mathsf{t_T})$ to $\mathcal{B}$. $\mathcal{B}$ answers $\perp$ $i = \perp$, and $\mathsf{m} = i - 1$, otherwise.

Challenge: $\mathcal{A}$ sends $(\mathsf{m}_0^*, \mathsf{m}_1^*) = (0, 1)$ and T^* to $\mathcal{B}$. $\mathcal{B}$ gives $(1, 2, T^*)$ to $\mathcal{C}$. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{b+1}, \mathsf{T}^*)$, and sends σ^* to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{C}^* = \sigma^*$ and sends C^* to $\mathcal{A}$.

Decryption Query (Phase II): $\mathcal{A}$ sends $(\mathsf{C}, \mathsf{T}) \neq (\mathsf{C}^*, T^*)$ to $\mathcal{B}$. $\mathcal{B}$ parses $\sigma = \mathsf{C}$ and sends the opening query (σ, T) to $\mathcal{C}$. $\mathcal{C}$ returns $i \leftarrow \mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{T}, \sigma, \mathsf{t_T})$ to $\mathcal{B}$. $\mathcal{B}$ answers $\perp$ $i = \perp$, and $\mathsf{m} = i - 1$, otherwise.

Guess: $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{B}$ outputs b' and breaks the $\mathsf{IND\text{-}TR\text{-}CCA_{TS}}$ security with the same advantage of $\mathcal{A}$.

□

Theorem 2. *The proposed construction provides the insider security if the underlying GS-MDO provides opener anonymity.*

Intuitively, due to opener anonymity, an opener with ok cannot obtain information of which signing key is used from group signatures unless they have a token corresponding to the message. Since the opener is set as the user in our TRE construction, the $\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$ adversary $\mathcal{A}$ that has $\mathsf{sk_u} = \mathsf{ok}$ cannot obtain information of plaintext from ciphertexts. Let $\mathcal{C}$ be the challenger of opener anonymity. We construct an algorithm $\mathcal{B}$ that breaks opener anonymity using $\mathcal{A}$ as follows. Here, $\mathcal{B}$ simulates the extract oracle using token generation oracle. When $\mathcal{A}$ sends an extract query $\mathsf{T} \neq \mathsf{T}^*$, $\mathcal{B}$ can simulate the extract oracle using the token generation oracle where the adversary is not allowed to send T^*.

Proof. We give the security proof as follows.

Setup: $\mathcal{C}$ computes $\mathsf{pp_{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \mathsf{gsk}_1, \mathsf{gsk}_2) \leftarrow \mathsf{GKg}(\mathsf{pp_{GS\text{-}MDO}}, 1^2)$, $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp_{GS\text{-}MDO}})$ and gives $(\mathsf{gpk}, \mathsf{ok}, \mathsf{apk}, \mathsf{gsk}_1, \mathsf{gsk}_2)$ to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{tpk} = \mathsf{apk}$, $\mathsf{pk_u} = (\mathsf{gpk}, \mathsf{gsk}_1, \mathsf{gsk}_2)$, $\mathsf{sk_u} = (\mathsf{pk_u}, \mathsf{ok})$ and gives $(\mathsf{tpk}, \mathsf{pk_u}, \mathsf{sk_u})$ to $\mathcal{A}$.

Extract Query (Phase I): $\mathcal{A}$ sends T to $\mathcal{B}$. $\mathcal{B}$ sends a token query T to $\mathcal{C}$. $\mathcal{C}$ returns $\mathsf{t_T}$ to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{s_T} = \mathsf{t_T}$ and sends $\mathsf{s_T}$ to $\mathcal{A}$.

Challenge: $\mathcal{A}$ sends $(\mathsf{m}_0^*, \mathsf{m}_1^*) = (0, 1)$, T^* to $\mathcal{B}$. $\mathcal{B}$ sends $(1, 2, T^*)$ to $\mathcal{C}$. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{b+1}, \mathsf{T}^*)$, and sends σ^* to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{C}^* = \sigma^*$ and sends C^* to $\mathcal{A}$.

Extract Query (Phase II): $\mathcal{A}$ sends $\mathsf{T} \neq \mathsf{T}^*$ to $\mathcal{B}$. $\mathcal{B}$ gives a token query T to $\mathcal{C}$. $\mathcal{C}$ returns $\mathsf{t_T}$ to $\mathcal{B}$. $\mathcal{B}$ sets $\mathsf{s_T} = \mathsf{t_T}$ and gives $\mathsf{s_T}$ to $\mathcal{A}$.

Guess: $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{B}$ outputs b' and breaks the $\mathsf{IND\text{-}TR\text{-}CPA_{IS}}$ security with the same advantage of $\mathcal{A}$.

□

6 Token Unforgeability

As mentioned in [35], a token $\mathsf{t_M}$ can be regarded as a signature since GS-MDO implies IBE and a decryption key of IBE can be regarded as a signature due to the Naor transformation [11]. For example, a token in the Ohara et al.'s pairing-based GS-MDO scheme [44] is a Boneh-Lynn-Shacham (BLS) signature [12], and a token in the Libert et al.'s lattice-based GS-MDO scheme [37] is a Gentry-Peikert-Vaikuntanathan (GPV) signature [31]. Thus, we expect that a kind of unforgeability

should be originally contained in the security definition of GS-MDO. In this section, we newly introduce token unforgeability and show that opener anonymity implies token unforgeability. In the security model, an adversary has all keys except ask and is allowed to issue token queries. This captures the situation that the opener receives a token $\mathsf{t_M}$ from the admitter and runs the Open algorithm to identify the signer who signed the message M. Token unforgeability guarantees that no opener can produce a token that the admitter did not generate.

Definition 7 (Token Unforgeability)**.** *Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger.*

Setup: *$\mathcal{C}$ computes $\mathsf{pp_{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \{\mathsf{gsk}_i\}_{i\in[n]}) \leftarrow \mathsf{GKg}(\mathsf{pp_{GS\text{-}MDO}}, 1^n)$, and $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp_{GS\text{-}MDO}})$, and gives $(\mathsf{gpk}, \mathsf{ok}, \mathsf{apk}, \{\mathsf{gsk}_i\}_{i\in[n]})$ to $\mathcal{A}$.*

Token Query: *$\mathcal{A}$ sends M to $\mathcal{C}$. If M is not previously queried, then $\mathcal{C}$ computes $\mathsf{t_M} \leftarrow \mathsf{Td}(\mathsf{ask}, \mathsf{M})$ and stores $(\mathsf{M}, \mathsf{t_M})$. Otherwise, $\mathcal{C}$ retrieves $\mathsf{t_M}$. $\mathcal{C}$ returns $\mathsf{t_M}$ to $\mathcal{A}$.*

Forge: *A outputs $(\mathsf{M}^*, \mathsf{t_{M^*}})$. $\mathcal{C}$ chooses $i \xleftarrow{\$} [n]$ and computes $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_i, \mathsf{M}^*)$. $\mathcal{A}$ wins if $\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{M}^*, \sigma^*, \mathsf{t_{M^*}}) = i$ holds and $\mathcal{A}$ did not send M^* as a token query.*

We say that GS-MDO *provides token unforgeability if, for all PPT adversaries $\mathcal{A}$, the advantage*

$$\mathsf{Adv}^{\mathsf{token\text{-}unforge}}_{\mathsf{GS\text{-}MDO},\mathcal{A}}(\lambda, n) = \Pr[\mathcal{A} \text{ wins}]$$

is negligible in the security parameter.

Next, we show that token unforgeability is implied by opener anonymity.

Theorem 3. *If a GS-MDO scheme provides opener anonymity, then it provides token unforgeability.*

Our security proof is inspired by the Naor transformation where a decryption key of IBE can be regarded as a signature. Informally, if an adversary of IBE, who is allowed to access the decryption key oracle, can produce a decryption key that the corresponding identity is not sent to the decryption key oracle, then the adversary breaks the IND-CPA security by decrypting the challenge ciphertext. In the following proof, the reduction algorithm $\mathcal{B}$ runs the Open algorithm against the challenge group signature by using the forged token given by the adversary $\mathcal{A}$, and breaks opneer anonymity.

Proof. Let $\mathcal{C}$ be the challenger of opener anonymity and $\mathcal{A}$ be an adversary of token unforgeability. We construct an algorithm $\mathcal{B}$ that breaks opener anonymity using $\mathcal{A}$ as follows.

Setup: $\mathcal{C}$ runs $\mathsf{pp_{GS\text{-}MDO}} \leftarrow \mathsf{GS\text{-}MDO.Setup}(1^\lambda)$, $(\mathsf{gpk}, \mathsf{ok}, \{\mathsf{gsk}_i\}_{i\in[n]}) \leftarrow \mathsf{GKg}(\mathsf{pp_{GS\text{-}MDO}}, 1^n)$, and $(\mathsf{apk}, \mathsf{ask}) \leftarrow \mathsf{AKg}(\mathsf{pp_{GS\text{-}MDO}})$, and gives $(\mathsf{gpk}, \mathsf{ok}, \mathsf{apk}, \{\mathsf{gsk}_i\}_{i\in[n]})$ to $\mathcal{B}$. $\mathcal{B}$ sends $(\mathsf{gpk}, \mathsf{ok}, \mathsf{apk}, \{\mathsf{gsk}_i\}_{i\in[n]})$ to $\mathcal{A}$.

Token Query: $\mathcal{A}$ sends M to $\mathcal{B}$. $\mathcal{B}$ forwards M to $\mathcal{C}$. $\mathcal{C}$ sends $\mathsf{t_M}$ to $\mathcal{B}$. $\mathcal{B}$ sends $\mathsf{t_M}$ to $\mathcal{A}$.

Forge: $\mathcal{A}$ outputs $(\mathsf{M}^*, \mathsf{t_{M^*}})$.

Since $\mathcal{A}$ did not send M^* to $\mathcal{B}$ as a token query, $\mathcal{B}$ did not send M^* to $\mathcal{C}$. $\mathcal{B}$ randomly chooses $i_0, i_1 \in [n]$ and sends (i_0, i_1, M^*) to $\mathcal{C}$ as the challenge. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0,1\}$, computes the challenge group signature $\sigma^* \leftarrow \mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_{i_b}, \mathsf{M}^*)$, and gives σ^* to $\mathcal{B}$. Since $\mathcal{A}$ can break token unforgeability, $\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, \mathsf{M}^*, \sigma^*, \mathsf{t}_{\mathsf{M}^*}) = i_b$ holds. $\mathcal{B}$ outputs b and breaks opener anonymity. □

On Strong Unforgeability: We note that $\mathcal{A}$ is not allowed to send M^* to the token oracle. One may think that the definition can be strengthen to capture a strong unforgeability flavor where for a set of queried messages and tokens that the oracle returns $\{(\mathsf{M}, \mathsf{t}_{\mathsf{M}})\}$, it is required that $(\mathsf{M}^*, \mathsf{t}_{\mathsf{M}^*}) \notin \{(\mathsf{M}, \mathsf{t}_{\mathsf{M}})\}$. Nevertheless, we follow the standard unforgeability flavor in this paper due to the following reasons. First, we recall that our motivation is to explicitly clarify a security property that GS-MDO originally achieved only implicitly. In this perspective, strong token unforgeability is not suitable because it is not directly implied by opener anonymity and is not originally achieved in GS-MDO. More precisely, let, $(\mathsf{M}^*, \mathsf{t}_{\mathsf{M}^*})$ be the the output of $\mathcal{A}$ where $\mathcal{A}$ has queried M^* to the token oracle and thus $\mathsf{t}_{\mathsf{M}^*}$ is not the response of the query. To simulate the token unforgeability game, the simulator $\mathcal{B}$ has forwarded M^* to the token oracle of the opener anonymity game. That is, $\mathcal{B}$ cannot set M^* as the challenge message and cannot break opener anonymity.

Second, we recall that if an IBE scheme provides a probabilistic key generation algorithm and produces decryption keys for the same ID, then these decryption keys work to decrypt a ciphertext encrypted by ID. Since a token is regarded as an IBE decryption key in the Sakai et al.'s IBE construction from GS-MDO, a token $\mathsf{t}'_{\mathsf{M}^*}$ of M^* where $\mathsf{t}'_{\mathsf{M}^*} \neq \mathsf{t}_{\mathsf{M}^*}$ is assumed to work for opening group signatures on M^* as well. The situation, that the opener tries to produce another token for a message that the corresponding token has been generated by the admitter, seems unnatural because the opener has already obtained a token that is sufficient to open group signatures. Thus, due to the theoretical and practical reasons as mentioned above, we follow the standard unforgeability flavor in this paper.

We note that some GS-MDO schemes may provide stronger token unforgeability, since the BLS signature scheme [12] and the GPV signature scheme [31], that are token of the Ohara et al.'s GS-MDO scheme [44] and the Libert et al.'s GS-MDO scheme [37], respectively, are strongly unforgeable. Exploring this direction is future work of this paper,

7 Conclusion

In this paper, we show that GS-MDO directly implies TRE. We also mentioned that the TRE scheme provides public verifiability. We argue that our result is evidence that GS-MDO is a strong cryptographic primitive. Our TRE construction is a combination of GS-based PKE [1] and GS-MDO-based IBE [47].

Our work introduces some open problems that may further strengthen the evidence that GS-MDO is a strong cryptographic primitive that we claimed in this paper.

On Traceability. We did not employ traceability which is also a fundamental security notion in GS-MDO. That is, there is a room for enhancing our result by providing traceability to construct a cryptographic primitive from GS-MDO. It is expected that the public verifiability guarantees that a valid TRE ciphertext is always decryptable if the underlying GS-MDO provides traceability. Providing a formal security definition of the public verifiability of GS-MDO-based TRE is left as future work, e.g., by extending the result in [27] that explores the public verifiability of GS-based PKE.

On Opening Soundness. We consider a static group setting as in the original GS-MDO definition [47]. In GS, Bellare, Shi, and Zhang (BSZ) [9] introduced a dynamic group setting and introduced the Judge algorithm that verifies a proof of ownership of a group signature. Later, Sakai et al. [48] introduced opening soundness as a security of the proof of ownership. Emura et al. [28] showed that dynamic GS implies PKENO and they employed opening soundness in their security proof. Due to these results and our result, we may be able to construct a TRE-variant of PKENO scheme from GS-MDO, e.g., by employing the Chen et al.'s dynamic GS-MDO scheme [18] that considers the Judge algorithm.

On Strongly Key-insulated Public Key Encryption. Our result connects GS-MDO with other cryptographic primitive. Cheon et al. [19] demonstrated that TRE is equivalent to strongly key-insulated public key encryption (SKIE) (with optimal threshold and random access key updates). In their TRE-to-SKIE construction, a SKIE ciphertext is a TRE ciphertext. Thus, if the underlying TRE scheme is publicly verifiable, then the SKIE scheme is also publicly verifiable. Note that a multiple-encryption technique [24] is employed in their SKIE-to-TRE construction. Thus, although SKIE might be constructed from GS-MDO, the TRE scheme constructed via the SKIE-to-TRE construction seems not publicly verifiable. Exploring the public verifiability of SKIE is also left as future work.

On Certificateless Encryption. In certificateless encryption (CLE) [4], the key generation center (KGC) issues a secret key for each user (as in IBE) and each user also manages own public and secret keys. As mentioned by Chow et al. [21], TRE is implicitly supported by a CLE mechanism. They also introduce a generalized model for CLE that fulfills the requirements of TRE. They mentioned that, the security of a user is preserved even if other users are compromised in a secure multi-user system, and a difference in the definitions of security between CLE and TRE where the public keys in TRE are certified while there is no certification in CLE, so public keys can be chosen adversarially. To support such adversarially chosen keys, their TRE model is somewhat different from ours, e.g., they introduced a partial decryption algorithm, a security-mediator, and a strong decryption oracle that works even if the public key is adversarially chosen but the secret key is not supplied. Thus, our result does not show that GS-MDO implies CLE (in the Chow et al.'s model). As another related work, Zhang et al. [49] proposed structure-preserving CLE and introduced Group Signatures with Certified Limited Opening (GS-CLO) as its application. GS-CLO can be seen as a generalization of GS-MDO. In GS-CLO, an entity called a master certifier is introduced, who certifies openers case by case depending on the context. These results suggest that there is a relation between GS-MDO and CLE. Exploring the implication between GS-MDO and CLE is also left as future work.

Acknowledgment: The authors thank Prof. Yohei Watanabe for his invaluable comments on the equivalence between SKIE and TRE. The authors also thank Scott Griffy for his insightful feedback and support. This work was supported by the Telecommunications Advancement Foundation and JSPS KAKENHI Grant Number JP21K11897.

References

[1] Michel Abdalla and Bogdan Warinschi. On the minimal assumptions of group signature schemes. In *ICICS*, pages 1–13, 2004.

[2] Masayuki Abe, Rosario Gennaro, and Kaoru Kurosawa. Tag-KEM/DEM: A new framework for hybrid encryption. *J. Cryptol.*, 21(1):97–130, 2008.

[3] Masayuki Abe, Fumitaka Hoshino, and Miyako Ohkubo. Design in type-I, run in type-III: Fast and scalable bilinear-type conversion using integer programming. In *CRYPTO*, pages 387–415, 2016.

[4] Sattam S. Al-Riyami and Kenneth G. Paterson. Certificateless public key cryptography. In *ASIACRYPT*, pages 452–473, 2003.

[5] Hiroaki Anada, Masayuki Fukumitsu, and Shingo Hasegawa. Dynamic group signatures with message dependent opening and non-interactive signing. In *CANDAR*, pages 76–82. IEEE, 2022.

[6] Hiromi Arai, Keita Emura, and Takuya Hayashi. Privacy-preserving data analysis: Providing traceability without big brother. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 104-A(1):2–19, 2021.

[7] James Bartusek, Sanjam Garg, Abhishek Jain, and Guru-Vamsi Policharla. End-to-End secure messaging with traceability only for illegal content. In *EUROCRYPT Part V*, pages 35–66, 2023.

[8] Mihir Bellare, Daniele Micciancio, and Bogdan Warinschi. Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In *EUROCRYPT*, pages 614–629, 2003.

[9] Mihir Bellare, Haixia Shi, and Chong Zhang. Foundations of group signatures: The case of dynamic groups. In *CT-RSA*, pages 136–153, 2005.

[10] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In *CRYPTO*, pages 41–55, 2004.

[11] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, pages 213–229, 2001.

[12] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.

[13] Dan Boneh, Periklis A. Papakonstantinou, Charles Rackoff, Yevgeniy Vahlis, and Brent Waters. On the impossibility of basing identity based encryption on trapdoor permutations. In *FOCS*, pages 283–292. IEEE Computer Society, 2008.

[14] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Essam Ghadafi, and Jens Groth. Foundations of fully dynamic group signatures. In *Applied Cryptography and Network Security*, pages 117–136, 2016.

[15] Xavier Boyen. Lattice mixing and vanishing trapdoors: A framework for fully secure short signatures and more. In *Public Key Cryptography*, pages 499–517, 2010.

[16] Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. In *EUROCRYPT*, pages 207–222, 2004.

[17] David Chaum and Eugène van Heyst. Group signatures. In *EUROCRYPT*, pages 257–265, 1991.

[18] Simin Chen, Jiageng Chen, Atsuko Miyaji, and Kaiming Chen. Constant-size group signatures with message-dependent opening from lattices. In *ProvSec*, pages 166–185, 2023.

[19] Jung Hee Cheon, Nicholas Hopper, Yongdae Kim, and Ivan Osipkov. Timed-release and key-insulated public key encryption. In *Financial Cryptography and Data Security*, pages 191–205, 2006.

[20] Jung Hee Cheon, Nicholas Hopper, Yongdae Kim, and Ivan Osipkov. Provably secure timed-release public key encryption. *ACM Transactions on Information and System Security*, 11(2):4:1–4:44, 2008.

[21] Sherman S. M. Chow, Volker Roth, and Eleanor Gilbert Rieffel. General certificateless encryption and timed-release encryption. In *SCN*, pages 126–143, 2008.

[22] Ivan Damgård, Dennis Hofheinz, Eike Kiltz, and Rune Thorbek. Public-key encryption with non-interactive opening. In *CT-RSA*, pages 239–255, 2008.

[23] Alexander W. Dent and Qiang Tang. Revisiting the security model for timed-release encryption with pre-open capability. In *ISC*, pages 158–174, 2007.

[24] Yevgeniy Dodis and Jonathan Katz. Chosen-ciphertext security of multiple encryption. In *TCC*, pages 188–209, 2005.

[25] Nico Döttling and Sanjam Garg. From selective IBE to full IBE and selective HIBE. In *TCC Part I*, pages 372–408, 2017.

[26] Nico Döttling and Sanjam Garg. Identity-based encryption from the Diffie-Hellman assumption. In *CRYPTO Part I*, pages 537–569, 2017.

[27] Keita Emura. On the traceability of group signatures: Uncorrupted user must exist. *IACR Cryptol. ePrint Arch.*, page 2014, 2024.

[28] Keita Emura, Goichiro Hanaoka, Yusuke Sakai, and Jacob C. N. Schuldt. Group signature implies public-key encryption with non-interactive opening. *International Journal of Information Security*, 13(1):51–62, 2014.

[29] Steven D. Galbraith, Kenneth G. Paterson, and Nigel P. Smart. Pairings for cryptographers. *Discret. Appl. Math.*, 156(16):3113–3121, 2008.

[30] David Galindo, Benoît Libert, Marc Fischlin, Georg Fuchsbauer, Anja Lehmann, Mark Manulis, and Dominique Schröder. Public-key encryption with non-interactive opening: New constructions and stronger definitions. In *AFRICACRYPT*, pages 333–350, 2010.

[31] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.

[32] Rishab Goyal, Venkata Koppula, and Mahesh Sreekumar Rajasree. A note on adaptive security in hierarchical identity-based encryption. In *CRYPTO*, 2025, to appear.

[33] Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In *EUROCRYPT*, pages 415–432, 2008.

[34] Yong Ho Hwang, Dae Hyun Yum, and Pil Joong Lee. Timed-release encryption with pre-open capability and its application to certified e-mail system. In *ISC*, pages 344–358, 2005.

[35] Yuto Imura and Keita Emura. An identity management system using group signatures with message-dependent opening. In *AsiaJCIS*, pages 40–47. IEEE, 2024.

[36] Benoît Libert and Marc Joye. Group signatures with message-dependent opening in the standard model. In *CT-RSA*, pages 286–306, 2014.

[37] Benoît Libert, Fabrice Mouhartem, and Khoa Nguyen. A lattice-based group signature scheme with message-dependent opening. In *ACNS*, pages 137–155, 2016.

[38] San Ling, Khoa Nguyen, and Huaxiong Wang. Group signatures from lattices: Simpler, tighter, shorter, ring-based. In *Public-Key Cryptography*, pages 427–449, 2015.

[39] San Ling, Khoa Nguyen, Huaxiong Wang, and Yanhong Xu. Lattice-based group signatures: Achieving full dynamicity with ease. In *Applied Cryptography and Network Security*, pages 293–312, 2017.

[40] Takahiro Matsuda, Yasumasa Nakai, and Kanta Matsuura. Efficient generic constructions of timed-release encryption with pre-open capability. In *Pairing-Based Cryptography*, pages 225–245, 2010.

[41] Steven A. Myers and Abhi Shelat. Bit encryption is complete. In *FOCS*, pages 607–616. IEEE, 2009.

[42] Yasumasa Nakai, Takahiro Matsuda, Wataru Kitada, and Kanta Matsuura. A generic construction of timed-release encryption with pre-open capability. In *IWSEC*, pages 53–70, 2009.

[43] Tuong Ngoc Nguyen, Willy Susilo, Dung Hoang Duong, Fuchun Guo, Kazuhide Fukushima, and Shinsaku Kiyomoto. A fault-tolerant content moderation mechanism for secure messaging systems. In *ACISP Part II*, pages 269–289, 2024.

[44] Kazuma Ohara, Yusuke Sakai, Keita Emura, and Goichiro Hanaoka. A group signature scheme with unbounded message-dependent opening. In *ACM ASIACCS*, pages 517–522, 2013.

[45] Go Ohtake, Arisa Fujii, Goichiro Hanaoka, and Kazuto Ogawa. On the theoretical gap between group signatures with and without unlinkability. In *AFRICACRYPT*, pages 149–166, 2009.

[46] Ronald L. Rivest, Adi Shamir, and David A. Wagner. Time-lock puzzles and timed-release crypto. In *Technical Report MITLCSTR-684, MIT*, 1996.

[47] Yusuke Sakai, Keita Emura, Goichiro Hanaoka, Yutaka Kawai, Takahiro Matsuda, and Kazumasa Omote. Group signatures with message-dependent opening. In *Pairing-Based Cryptography*, pages 270–294, 2012.

[48] Yusuke Sakai, Jacob C. N. Schuldt, Keita Emura, Goichiro Hanaoka, and Kazuo Ohta. On the security of dynamic group signatures: Preventing signature hijacking. In *Public Key Cryptography*, pages 715–732, 2012.

[49] Tao Zhang, Huangting Wu, and Sherman S. M. Chow. Structure-preserving certificateless encryption and its application. In *CT-RSA*, pages 1–22, 2019.

Appendix

A Modified Ohara et al.'s GS-MDO scheme

Here, we give the modified Ohara et al.'s GS-MDO scheme. The original Ohara et al.'s GS-MDO scheme [44] can be seen as a combination of the Boneh-Boyen-Shacham (BBS) group signature

scheme [10] and the Boneh-Franklin (BF) IBE scheme [11]. A token is a decryption key of the BF IBE scheme and thus it can be regarded as a Boneh-Lynn-Shacham (BLS) signature [12]. The original Ohara et al.'s GS-MDO scheme is constructed on symmetric pairings. However, asymmetric pairings are suitable for providing an efficient implementation [29]. Arai et al. [6] employed the Abe-Hoshino-Ohkubo transformation [3], that transforms a scheme constructed using a symmetric pairing to a scheme constructed using an asymmetric pairing, to the Ohara et al. GS-MDO scheme. Since the Arai et al.'s variant of the Ohara et al's GS-MDO scheme follows the original syntax defined in [47], we modify it to match the syntax defined in Def. 1 as follows. Let p is a λ-bit prime, $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are groups of order p, $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a bilinear map, and g_1 and g_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. We require bilinearlity: for all $h_1 \in \mathbb{G}_1$, $h_2 \in \mathbb{G}_2$, and $a, b \in \mathbb{Z}_p$, $e(h_1^a, h_2^b) = e(h_1, h_2)^{ab}$ holds, and non-degeneracy: $e(g_1, g_2) \neq 1$ holds.

$\mathsf{GS\text{-}MDO.Setup}(1^\lambda)$: Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ be an asymmetric bilinear group where $\langle g_1 \rangle = \mathbb{G}_1$ and $\langle g_2 \rangle = \mathbb{G}_2$. Let $H_1 : \{0,1\}^* \to \mathbb{G}_2$ and $H_2 : \{0,1\}^* \to \mathbb{Z}_p$ be hash functions (modeled as random oracles). Output $\mathsf{pp_{GS\text{-}MDO}} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, H_1, H_2)$.

$\mathsf{GKg}(\mathsf{pp_{GS\text{-}MDO}}, 1^n)$**:** Parse $\mathsf{pp_{GS\text{-}MDO}} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, H_1, H_2)$.

1. Choose $u, v, h \xleftarrow{\$} \mathbb{G}_1$ and $\xi_1, \xi_2, \xi_3, \gamma \xleftarrow{\$} \mathbb{Z}_p$.
2. Compute $\bar{g}_1 = u^{\xi_1} v^{\xi_3}$, $\bar{g}_2 = v^{\xi_2} h^{\xi_3}$, and $\omega = g_2^\gamma$.
3. For $i \in [1, n]$, choose $x_i \xleftarrow{\$} \mathbb{Z}_p$, compute $A_i = g_1^{1/(\gamma + x_i)}$, and set $\mathsf{gsk}_i = (A_i, x_i)$.

Output $\mathsf{gpk} = ((p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2), u, v, h, \bar{g}_1, \bar{g}_2, \omega, H_1, H_2)$, $\{\mathsf{gsk}_i\}_{i \in [n]}$, and $\mathsf{ok} = (\xi_1, \xi_2, \xi_3, \{e(A_i, g_2)\}_{i \in [1,n]})$.

$\mathsf{AKg}(\mathsf{pp_{GS\text{-}MDO}})$**:** Parse $\mathsf{pp_{GS\text{-}MDO}} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, H_1, H_2)$.

1. Choose $\xi \xleftarrow{\$} \mathbb{Z}_p$.
2. Compute $y = g_1^\xi$.

Output $\mathsf{apk} = y$ and $\mathsf{ask} = \xi$.

$\mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_i, \mathsf{M})$**:** Parse $\mathsf{gpk} = ((p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2), u, v, h, \bar{g}_1, \bar{g}_2, \omega, H_1, H_2)$, $\mathsf{apk} = y$, and $\mathsf{gsk}_i = (A_i, x_i)$.

1. Choose $\alpha, \beta, \rho, \eta \xleftarrow{\$} \mathbb{Z}_p$.
2. Compute

$$T_1 = u^\alpha, T_2 = v^\beta, T_3 = h^{\alpha+\beta}, T_4 = \bar{g}_1^\alpha \bar{g}_2^\beta A_i g_1^\eta$$
$$T_5 = g_1^\rho, T_6 = e(y, H_1(\mathsf{M}))^\rho e(g_1, g_2)^{-\eta}$$

3. Choose $r_\alpha, r_\beta, r_\rho, r_\eta, r_x, r_{\alpha x}, r_{\beta x}, r_{\rho x}, r_{\eta x} \xleftarrow{\$} \mathbb{Z}_p$.
4. Compute

$$R_1 = u^{r_\alpha}, R_2 = v^{r_\beta}, R_3 = h^{r_\alpha + r_\beta},$$
$$R_4 = e(T_4, g_2)^{r_x} e(g_1, \omega)^{-r_\alpha} e(\bar{g}_1, g_2)^{-r_{\alpha x}} e(\bar{g}_2, \omega)^{-r_\beta} e(\bar{g}_2, g_2)^{-r_{\beta x}} e(g_1, \omega)^{-r_\eta} e(g_1, g_2)^{-r_{\eta x}},$$
$$R_5 = g_1^{r_\rho}, R_6 = e(y, H_1(\mathsf{M}))^{r_\rho} e(g_1, g_2)^{-r_\eta}, R_7 = T_1^{r_x} u^{-r_{\alpha x}}, R_8 = T_2^{r_x} v^{-r_\beta}, R_9 = T_5^{r_x} g_1^{-r_\rho}$$
$$R_{10} = T_6^{r_x} e(y, H_1(\mathsf{M}))^{-r_\rho} e(g_1, g_2)^{r_{\eta x}}.$$

5. Compute $c = H_2(M, T_1, \ldots, T_6, R_1, \ldots, R_{10})$, $s_\alpha = r_\alpha + c\alpha$, $s_\beta = r_\beta + c\beta$, $s_\rho = r_\rho + c\rho$, $s_\eta = r_\eta + c\eta$, $s_x = r_x + cx_i$, $s_{\alpha x} = r_{\alpha x} + c\alpha x_i$, $s_{\beta x} = r_{\beta x} + c\beta x_i$, $s_{\rho x} = r_{\rho x} + c\rho x_i$, and $s_{\eta x} = r_{\eta x} + c\eta x_i$.

Output $\sigma = (T_1, \ldots, T_6, c, s_\alpha, s_\beta, s_\rho, s_\eta, s_x, s_{\alpha x}, s_{\beta x}, s_{\rho x}, s_{\eta x})$.

$\mathsf{Td}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, \mathsf{ask}, \mathsf{M})$**:** Parse $\mathsf{pp}_{\mathsf{GS\text{-}MDO}} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, H_1, H_2)$ and $\mathsf{ask} = \xi$. Output $\mathsf{t}_\mathsf{M} = H_1(\mathsf{M})^\xi$.

$\mathsf{GVf}(\mathsf{gpk}, \mathsf{apk}, \mathsf{M}, \sigma)$**:** Parse $\mathsf{gpk} = ((p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2), u, v, h, \bar{g}_1, \bar{g}_2, \omega, H_1, H_2)$, $\mathsf{apk} = y$, and $\sigma = (T_1, \ldots, T_6, c, s_\alpha, s_\beta, s_\rho, s_\eta, s_x, s_{\alpha x}, s_{\beta x}, s_{\rho x}, s_{\eta x})$. Compute

$$\begin{aligned}
R'_1 &= u^{s_\alpha} T_1^{-c}, R'_2 = v^{s_\beta} T_2^{-c}, R'_3 = h^{s_\alpha + s_\beta} T_3^{-c}, \\
R'_4 &= e(T_4, g_2)^{s_x} e(\bar{g}_1, \omega)^{-s_\alpha} e(\bar{g}_1, g_2)^{-s_{\alpha x}} e(\bar{g}_2, \omega)^{-s_\beta} e(\bar{g}_2, g_2)^{-s_{\beta x}} e(g_1, \omega)^{-s_\eta} \\
&\quad \times e(g_1, g_2)^{-s_{\eta x}} (e(g_1, g_2)/e(T_4, \omega))^{-c}, \\
R'_5 &= g_1^{s_\rho} T_5^{-c}, R'_6 = e(y, H_1(\mathsf{M}))^{s_\rho} e(g_1, g_2)^{-s_\eta} T_6^{-c}, \\
R'_7 &= T_1^{s_x} u^{-s_{\alpha x}}, R'_8 = T_2^{s_x} v^{-s_{\beta x}}, R'_9 = T_5^{s_x} g_1^{-s_{\rho x}}, \\
R'_{10} &= T_6^{s_x} e(y, H_1(\mathsf{M}))^{-s_{\rho x}} e(g_1, g_2)^{s_{\eta x}}
\end{aligned}$$

If $c = H_2(M, T_1, \ldots, T_6, R'_1, \ldots, R'_{10})$, then return 1, and 0 otherwise.

$\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, M, \sigma, \mathsf{t}_\mathsf{M})$**:** Parse $\mathsf{gpk} = ((p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2), u, v, h, \bar{g}_1, \bar{g}_2, \omega, H_1, H_2)$, $\mathsf{apk} = y$, $\mathsf{ok} = (\xi_1, \xi_2, \xi_3, \{e(A_i, g_2)\}_{i \in [1,n]})$, and $\sigma = (T_1, \ldots, T_6, c, s_\alpha, s_\beta, s_\rho, s_\eta, s_x, s_{\alpha x}, s_{\beta x}, s_{\rho x}, s_{\eta x})$. If $\mathsf{GVf}(\mathsf{gpk}, \mathsf{apk}, M, \sigma) = 0$, then output $\perp$. Otherwise, If $\exists i \in [1, n]$ s.t.

$$e(T_4 / T_1^{\xi_1} T_2^{\xi_2} T_3^{\xi_3}, g_2) \cdot T_6 / e(T_5, \mathsf{t}_\mathsf{M}) = e(A_i, g_2)$$

then output i. Otherwise, output $\perp$.

B Modified Libert et al.'s GS-MDO scheme

The Libert et al.'s GS-MDO scheme can be seen as a combination of the Lin-Nguyen-Wang (LNW) group signature scheme [38] and the GPV IBE scheme [31]. A group signing key is a Boyen's signature [15]. A token is a decryption key of the GPV IBE scheme and thus it can be regarded as a GPV signature. In the Libert et al.'s GS-MDO scheme, some lattice-related parameters depend on the number of group members $2^\ell = \mathsf{poly}(\lambda)$. That is, $\mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^n)$ needs to indicate these parameters since the algorithm takes the number of group members n as input. Moreover, the AKg algorithm takes these parameters as input. This fact prevents to directly modify the syntax. One simple solution is that the maximum number of group members N_max is set in the $\mathsf{GS\text{-}MDO.Setup}$ algorithm. Then, we can modify the syntax. The maximum number of group users N_max is often set to specify the lattice-related parameters, e.g., the Lin et al.'s lattice-based fully dynamic group signature scheme [39]. For consistency, we denote $n \leq N_\mathsf{max}$ as the number of group members, though n is often employed to indicate a lattice parameter such as $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. We denote N as the lattice parameter.

$\mathsf{GS\text{-}MDO.Setup}(1^\lambda)$: Choose the lattice parameter $N = O(\lambda)$. Set the maximum number of group members $N_\mathsf{max} = 2^\ell = \mathsf{poly}(\lambda)$, a prime modulus $q = O(\ell \cdot N^2)$, the infinity norm bound for

signatures (generated by the Boyen's signature scheme) $\beta = \tilde{O})(\sqrt{\ell N})$, $m \leq \mathsf{poly}(N)$, and the infinity norm bound for LWE noises sampled from error distribution χ. Assume that $\mathsf{pp}_{\mathsf{GS\text{-}MDO}}$ contains these lattice parameters, N_{max}, two hash functions $H_1 : \{0,1\} \to \mathbb{Z}_q^{N \times \ell}$ and $H_2 : \{0,1\} \to \mathbb{Z}_q^{N \times \ell \lceil \log q \rceil}$ and a strongly unforgeable one-time signature scheme $\Pi_{\mathsf{OTS}} = (\mathsf{OTS.KeyGen}, \mathsf{OTS.Sign}, \mathsf{OTS.Verify})$. We denote $(\mathsf{vk}, \mathsf{sigk})$ be a key pair of Π_{OTS}.

$\mathsf{GKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, 1^n)$**:** Assume that $n \leq N_{\mathsf{max}}$. Generate a key pair of the Boyen's signature scheme: a verification key $(\mathbf{A}, \mathbf{A}_0, \ldots, \mathbf{A}_\ell, \mathbf{u}) \in (\mathbb{Z}_q^{N \times m})^{\ell+2}$ and a signing key $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}^{m \times m}$. For each $d \in \{0, \ldots, 2^{\ell-1}\}$, generate the Boyen's signature for the message $\mathsf{bin}(d) = (d_1, \ldots, d_\ell) \in \{0,1\}^\ell$ using the trapdoor $\mathbf{T}_{\mathbf{A}}$ such that $\mathsf{gsk}_d = (\mathbf{v}_{d,1}^T \mid \mathbf{v}_{d,2}^T) \in \mathbb{Z}^{2m}$. Generate an encryption key $\mathbf{B} \in \mathbb{Z}_q^{N \times m}$ and a decryption key $\mathbf{T}_{\mathbf{B}} \in \mathbb{Z}^{m \times m}$ of the GPV-IBE scheme. Output $\mathsf{gpk} = (\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, \mathbf{A}, \{\mathbf{A}_i\}_{i=0}^{\ell}, \mathbf{u}, \mathbf{B})$, $\{\mathsf{gsk}_i\}_{i \in [n]}$, and $\mathsf{ok} = \mathbf{T}_{\mathbf{B}}$.

$\mathsf{AKg}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}})$**:** Generate an encryption key $\mathbf{C} \in \mathbb{Z}_q^{N \times m}$ and a decryption key $\mathbf{T}_{\mathbf{C}} \in \mathbb{Z}^{m \times m}$ of the GPV-IBE scheme. Output $\mathsf{apk} = \mathbf{C}$ and $\mathsf{ask} = \mathbf{T}_{\mathbf{C}}$.

$\mathsf{GSig}(\mathsf{gpk}, \mathsf{apk}, \mathsf{gsk}_i, \mathsf{M})$**:** Generate $(\mathsf{vk}, \mathsf{sigk}) \leftarrow \mathsf{OTS.KeyGen}(1^\lambda)$. Compute $\mathbf{G} = H_1(\mathsf{vk})$ and a GPV ciphertext (with vk as the identity for the message i) $(\mathbf{c}_1 = \mathbf{B}^T\mathbf{s} + \mathbf{e}_1, \mathbf{c}_2 = \mathbf{G}^T\mathbf{s} + \mathbf{e}_2 + \lfloor q/2 \rfloor \cdot \mathsf{bin}(i)) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^\ell$ where $\mathbf{s}$, $\mathbf{e}_1$, and $\mathbf{e}_2$ are appropriately sampled. Compute $\hat{\mathbf{G}} = H_2(\mathsf{M})$ and a GPV ciphertext (with M as the identity for the message $\mathbf{c}_2$) $(\hat{\mathbf{c}}_1 = \mathbf{C}^T\hat{\mathbf{s}} + \hat{\mathbf{e}}_1, \hat{\mathbf{c}}_2 = \hat{\mathbf{G}}^T\hat{\mathbf{s}} + \hat{\mathbf{e}}_2 + \lfloor q/2 \rfloor \cdot \mathsf{bin}(\mathbf{c}_2)) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{\ell \lceil \log q \rceil}$ where $\hat{\mathbf{s}}$, $\hat{\mathbf{e}}_1$, and $\hat{\mathbf{e}}_2$ are appropriately sampled. Generate a NIZKAoK (zero-knowledge arguments of knowledge) π for the relation R_{gsmdo} and convert via the Fiat-Shamir transformation (See [37] for detail). Compute a OTS Σ for $(\mathbf{c}_1, \hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2)$ and π. Output $\sigma = (\mathsf{vk}, \Sigma, \mathbf{c}_1, \hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2, \pi)$.

$\mathsf{Td}(\mathsf{pp}_{\mathsf{GS\text{-}MDO}}, \mathsf{ask}, \mathsf{M})$**:** Compute $\hat{\mathbf{G}} = H_2(\mathsf{M})$. Using $\mathbf{T}_{\mathbf{C}}$, compute a small-norm matrix $\mathbf{E}_{\mathsf{M}}$ satisfying $\mathbf{C} \cdot \mathbf{E}_M = \hat{\mathbf{G}}$ which is a decryption key of the GPV-IBE scheme for the identity M. Output $\mathsf{t}_{\mathsf{M}} = \mathbf{E}_{\mathsf{M}}$.

$\mathsf{GVf}(\mathsf{gpk}, \mathsf{apk}, \mathsf{M}, \sigma)$**:** Output 1 of the OTS Σ and the proof π are correct. Otherwise, output 0.

$\mathsf{Open}(\mathsf{gpk}, \mathsf{apk}, \mathsf{ok}, M, \sigma, \mathsf{t}_{\mathsf{M}})$**:** Decrypt $(\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2)$ using $\mathsf{t}_{\mathsf{M}} = \mathbf{E}_{\mathsf{M}}$ and obtain $\mathbf{c}_2$. Decrypt $(\mathbf{c}_1, \mathbf{c}_2)$ using $\mathsf{ok} = \mathbf{T}_{\mathbf{B}}$ and obtain i. Output i if $i \in [n]$ and $\perp$ otherwise.

C On Multiple Openers and Users

C.1 GS-MDO in a Multi-Opener Setting

Let $\{\mathsf{gpk}_i, \mathsf{ok}_i\}_{i \in [\ell]}$ be the set of openers' keys where $\ell \in \mathbb{N}$ denotes the number of openers. First of all, an adversary must obtain $\{\mathsf{gpk}_i\}_{i \in [\ell]}$. The situations we need to capture in a multi-opener setting are (1) an adversary of opener anonymity obtains all openers' secret keys that can be easily handled, and (2) an adversary of admitter anonymity obtains all openers' secret keys except the challenge opener. In an adaptive security manner, the adversary may corrupt up to $\ell - 1$ openers and obtain the corresponding ok. With the reduction loss $1/\ell$, the final (i.e., challenge) opener can be guessed. This implies that we can follow a selective security manner, where an adversary declares who will be the challenge opener, with the reduction loss $1/\ell$. We assume that the adversary issues open queries for the challenge opener since the adversary can run the Open algorithm using

other openers' secret keys. The remaining procedure follows that of admitter anonymity in the single-opener setting as defined in Def. 2.

We also remark that an adversary of traceability, that is not required to construct TRE, receives ok in the security model. As with opener anonymity, traceability in the multi-opener setting can be handled by providing all openers' secret keys to the adversary.

C.2 TRE in a Multi-User Setting

Let $\{(\mathsf{pk}_{\mathsf{u},i}, \mathsf{sk}_{\mathsf{u},i})\}_{i\in[\ell]}$ be the set of users' keys where $\ell \in \mathbb{N}$ denotes the number of users. First of all, an adversary must obtain $\{\mathsf{pk}_{\mathsf{u},i}\}_{i\in[\ell]}$. The situations we need to capture in a multi-user setting are (1) an adversary of insider security obtains all users' secret keys $\{\mathsf{sk}_{\mathsf{u},i}\}_{i\in[\ell]}$ that can be easily handled, and (2) an adversary of time server security obtains all users' secret keys except the challenge user. In an adaptive security manner, the adversary may corrupt up to $\ell - 1$ users and obtain the corresponding sk_{u}. With the reduction loss $1/\ell$, the final (i.e., challenge) user can be guessed. This implies that we can follow a selective security manner, where an adversary declares who will be the challenge user, with the reduction loss $1/\ell$. We assume that the adversary issues decryption queries for the challenge user since the adversary can run the $\mathsf{TRE.Dec}$ algorithm using other users' secret keys. The remaining procedure follows that of time server security in the single-user setting as defined in Def. 6.

C.3 Remark on Security Proofs

Security proofs of Theorem 1 and Theorem 2 are essentially the same even in a multi-user setting. To reduce admitter anonymity to time server security, the reduction algorithm obtains all openers' secret keys from the challenger of admitter anonymity in the multi-opener setting. Thus, the reduction algorithm simply forwards these keys to the adversary of time server security. To reduce opener anonymity to insider security, the reduction algorithm obtains all openers' secret keys, except the challenge opener's secret key, from the challenger of opener anonymity in the multi-opener setting. Since the adversary of time server security declares who the challenge user is, the reduction algorithm assigns the challenge opener as the challenge user.